

Library Management System

Author: Itumeleng Mphuti

Document purpose: To identify, categorize, and prioritize issues found during reviewing and testing the library management code.

1. HTML file:

a. Critical:

- Tab navigation has no ARIA roles - #catalogue-tab, #member-tab, and #statistics-tab implement a tab pattern but lack the appropriate ARIA roles, such as role="tablist", role="tab", aria-selected, and aria-controls. As a result, screen reader users cannot properly perceive or operate this navigation.
- Form inputs have no associated labels - The #search, #filter-category, #member-id, and #isbn inputs rely solely on placeholder text and do not have associated <label> elements. This reduces accessibility for screen reader users and negatively impacts form usability.

b. Major:

- Missing <meta> tags - The <head> section does not contain a description or other metadata tags. This negatively affects search engine optimization (SEO) and how the page is represented in search engines and browsers.
- Non-semantic HTML structure - <div> elements are used throughout the document where semantic elements such as <section>, <article>, or similar tags would better communicate the document structure to assistive technologies.
- Dynamic content regions lack aria-live - The #catalogue-list and #book-details elements update dynamically but do not include an aria-live region. As a result, screen reader users are not notified when content changes.
- Inline styles used for visibility control - The style="display: none;" attribute is applied to #member-section and #statistics-section,

mixing presentation with markup. Visibility should instead be controlled by toggling a CSS class.

c. Minor:

- Tab buttons are missing an explicit type="button" - Although this is currently harmless because the buttons are outside a form, it could become a bug if the markup changes in the future.
- No input constraints on the ISBN field - The ISBN input is missing attributes such as pattern, inputmode, or maxlength. Validation currently relies entirely on JavaScript.
- No autocomplete attributes on form fields - Form fields do not use the autocomplete attribute, reducing user convenience and browser-assisted form completion.
- No skip-to-content link for keyboard users - The page does not provide a skip-to-content link, making keyboard navigation less accessible
- No favicon or theme-color meta tag - The document does not include a favicon or a theme-color meta tag, reducing browser integration and branding.

d. Enhancement/ Modernization notes

- Hardcoded repeated markup - The statistic cards and, eventually, the book list items are written as static HTML instead of being generated dynamically. Consider using a <template> element or JavaScript-driven rendering after reviewing the JavaScript files.
- No validation or error feedback container on #borrow-form - There is no dedicated container for displaying validation or error messages, such as "Member not found." A feedback element should be added to improve the user experience.
- Scripts are loaded in the incorrect order and without type="module" - The JavaScript files are loaded without using type="module", and their load order creates unnecessary dependencies between

scripts. Converting them to ES modules would improve maintainability and reduce load-order issues.

2. JavaScript: [library.js](#)

a. Critical

- Undeclared global variables - The `books = []` and `MAX_BOOKS_PER_MEMBER = 5` variables are assigned without a declaration keyword. As a result, they become implicit global variables, polluting the window object and throwing a `ReferenceError` in strict mode. `books` should be declared using `let`, while `MAX_BOOKS_PER_MEMBER` should be declared using `const`.
- Assignment operator used instead of a comparison operator - The assignment operator (`=`) is used instead of the strict equality operator (`===`) in three places:
 - `canBorrow()` uses `this.borrowedBooks.length = MAX_BOOKS_PER_MEMBER`, which assigns a value and always evaluates as truthy.
 - `findMemberById()` uses `members[i].id = id`, which assigns the value instead of comparing it.
 - `searchBooksByCategory()` uses `bookList[index].category = category`, which also performs an assignment rather than a comparison. (All these instances should use the strict equality operator `===`).
- `searchBooksByCategory()` has no base case - The recursive function does not check whether `index` has exceeded `bookList.length`. This results in infinite recursion and eventually causes a stack overflow whenever the function is executed.
- `super()` is missing in `DigitalBook` - The constructor accesses `this.fileSize` and `this.format` before calling `super()`. This causes a `ReferenceError` at runtime because this cannot be accessed until the parent constructor has been invoked.

- `borrowBook()` has no null or undefined guard - Both `findMemberById()` and `findBookByISBN()` can return null or undefined. Calling `member.canBorrow()` or `book.checkOut()` on these values causes an immediate runtime error. There is also no `try...catch` block to handle these exceptions gracefully.

b. Major

- `members` should not be declared with `var` - The `members` array is declared using `var`, which is function-scoped and hoisted. This makes the declaration less predictable. Since the array is reassignable, it should be declared using `let`.
- `var` is used throughout the file - Every variable is declared using `var`. Variables whose values never change should use `const`, while variables that are reassigned should use `let`. Updating the code to use modern declarations will improve readability and reduce potential bugs.
- `Book` is missing `availableCopies` and `totalCopies` - The `Book` constructor does not define `availableCopies` or `totalCopies`. Consequently, `checkOut()` allows unlimited borrowing because there is no mechanism for tracking available stock.
- `Member` is missing `joinDate` - The `Member` constructor does not include a `joinDate` property. Without this information, it is impossible to calculate membership duration or manage membership expiry.
- `findMemberById()` implicitly returns undefined - When no matching member is found, the function reaches the end without returning a value. As a result, `borrowBook()` receives undefined and attempts to call methods on it, causing runtime errors.
- `borrowBook()` has no `try...catch` block - Any failure, such as a missing member, a missing book, or future asynchronous errors, results in an unhandled exception with no feedback provided to the user interface.

- `getBooksByAuthor()` uses `==` instead of `===` - The function uses loose equality (`==`), which allows type coercion. Strict equality (`===`) should be used consistently throughout the application.
- `searchBooksByCategory()` has no null or undefined check - The function accesses `bookList[index]` without verifying that the element exists. Combined with the missing base case, this increases the likelihood of runtime errors.
- Missing module exports - No export statements are defined anywhere in the file. As a result, `ui.js` and other modules cannot import the classes or functions when using ES modules. The current implementation also makes the HTML scripts dependent on their loading order.

c. Minor

- Replace for loops with array methods - Several traditional for loops should be replaced with modern array methods to improve readability and maintainability:
 - `getBooksByAuthor()` should use `filter()`.
 - `findMemberById()` should use `find()`.
 - `calculateTotalLateFees()` should use `reduce()`.
 - `getMostPopularBook()` in `LibraryStats` should use `reduce()`.
- `combineBookCollections()` should use the spread operator - The function currently uses three separate for loops to merge arrays. This is unnecessarily verbose. The following implementation is more concise and idiomatic:


```
return [...fiction, ...nonFiction, ...reference];
```
- `addMultipleBooks()` should use rest parameters - The function currently accepts exactly three named parameters. Using a rest parameter (`...books`) allows any number of books to be added in a single call.

- `updateMemberInfo()` should use destructuring - The function manually extracts each property from the `updates` object. Destructuring makes the expected object structure clearer and reduces repetitive code.
- `formatBookInfo()` should use template literals - The function uses string concatenation across multiple lines, making it difficult to read and maintain. Template literals provide a cleaner and more modern alternative
- `calculateFineAmount()` is missing `toFixed(2)` - Floating-point calculations such as `daysLate * 0.50` can produce values like `1.5000000000000002`. Currency values should be formatted using `.toFixed(2)` before being displayed or stored.
- `calculateFineAmount()` has no type or NaN validation - If `daysLate` is undefined, null, or a non-numeric string, the result becomes NaN without any indication that an error has occurred.

d. Enhancement/ Modernisation notes

- No Map or Set is used for lookups - `findBookByISBN()` searches the entire `books` array using a while loop, resulting in $O(n)$ lookup time. A Map keyed by ISBN would provide $O(1)$ lookup time and is the more appropriate data structure.
- `PremiumMember` never overrides `canBorrow()` - Although the class exists, it does not provide any unique behaviour. Premium members are currently restricted by the same borrowing limit as standard members.
- `DigitalBook.download()` does not meaningfully override `checkOut()` - The method increments a counter but never updates `availableCopies` or performs any validation. This causes digital and physical borrowing behaviour to be inconsistent.
- `LibraryStats` should be a class - `LibraryStats` is currently implemented as an object literal despite containing both state (`totalBooks`, `totalMembers`) and behaviour (`updateStats()`),

getMostPopularBook()). Refactoring it into a class would improve extensibility, testability, and maintainability.

- Missing methods referenced in comments - Several methods referenced in comments have not been implemented:
 - Book is missing an availability check method.
 - Book is missing a template literal information method.
 - Member is missing a membership duration calculator.
 - Member is missing a destructuring-based update method.
 - LibraryStats is missing a Math-based calculation method.
 - LibraryStats is missing a for...of loop implementation.

These methods are referenced in the code comments but have not been implemented.

3. JavaScript: [ui.js](#)

a. Critical

- initializeUI() is called before the DOM is ready - The initializeUI() function is invoked at the bottom of the file before the browser has finished parsing the HTML document. As a result, every document.querySelector() and getElementById() call inside the function returns null, causing the UI initialization process to fail silently. The function should be wrapped inside a DOMContentLoaded event listener to ensure that the DOM has been fully loaded before any elements are accessed.
- filterDropdown selector is missing the # - The statement document.querySelector("filter-category") attempts to select an HTML element named filter-category, which does not exist. Because the selector is missing the #, filterDropdown is assigned null, causing every subsequent reference to it to throw a TypeError. The selector should be changed to document.querySelector("#filter-category").
- handleFilterChange() uses assignment instead of comparison - The handleFilterChange() function uses the assignment operator (=)

instead of the strict equality operator (`===`) when comparing book categories. This causes every book's category to be overwritten with the selected category, corrupting the application's data.

Replacing the assignment operator with `===` will correctly compare the values without modifying the data.

- `handleBorrowSubmit()` is missing `event.preventDefault()` - The form submission handler does not call `event.preventDefault()`.
Consequently, submitting the borrow form triggers a full page reload, causing all in-memory data, including the books and members arrays, to be lost. Preventing the default form submission allows JavaScript to process the request without refreshing the page.
- No null checks on DOM queries - The variables `catalogueContainer`, `searchInput`, `filterDropdown`, and `borrowForm` are used immediately after being queried without verifying that the corresponding DOM elements exist. If any of these elements are missing due to a typo or structural change in the HTML, the initialization process throws an exception and leaves the interface unusable. Each queried element should be validated before use.
- `saveToLocalStorage()` stores raw arrays without `JSON.stringify()` - The `saveToLocalStorage()` function stores the books and members arrays directly in local storage. Since `localStorage.setItem()` automatically converts values to strings, the arrays are saved as `"[object Object]"`, making the stored data unreadable and impossible to restore correctly. The arrays should be converted to JSON using `JSON.stringify()` before storage.
- `loadFromLocalStorage()` assigns raw strings without `JSON.parse()` - The `loadFromLocalStorage()` function assigns the strings returned by `localStorage.getItem()` directly to the books and members variables. As a result, the arrays are replaced with plain strings, causing methods such as `books.length` and `members.push()` to

behave incorrectly or fail completely. The stored JSON should be converted back into JavaScript objects using `JSON.parse()`.

b. Major

- Global UI variables are declared using `var` - The variables `catalogueContainer`, `searchInput`, and `filterDropdown` are declared using `var`, making them globally scoped and hoisted. Since these variables are only used within the UI initialization process, they should be declared as `const` inside the `initializeUI()` function.
- `filterDropdown` uses the wrong event type - A "click" event listener is attached to the `<select>` element. The correct event for detecting changes to a dropdown menu is "change", ensuring that the filtering logic executes whenever the selected option changes..
- No search event listener is registered - Although the `handleSearch()` function exists, it is never attached to `searchInput`. Consequently, entering text into the search field has no effect. A suitable event listener should be registered so that searches are processed as the user types or submits a query.
- `renderBookCatalogue()` never clears the container - Each time the catalogue is rendered, new book cards are appended to the existing content rather than replacing it. This causes duplicate entries to accumulate whenever the catalogue is filtered or searched. The container should be cleared before rendering a new list of books..
- `renderBookCatalogue()` repeatedly updates `innerHTML` - The function repeatedly reads and reassigns `bookCard.innerHTML` inside a loop. Every reassignment forces the browser to parse the HTML again and rebuild the element's contents, reducing performance. Building a single template literal and assigning it once is significantly more efficient.
- `handleBookClick()` does not use `event.target.closest()` - The clicked element may be a child element, such as an `<h3>` or `<p>`, rather

than the book card itself. As a result, `bookElement.id` may be empty or undefined. Using `event.target.closest(".book-card")` ensures that the correct card element is always selected.

- No event delegation for dynamically rendered book cards - Click event listeners would need to be attached every time the catalogue is re-rendered and removed when the content changes. Using event delegation by attaching a single listener to `#catalogue-list` is more efficient and easier to maintain.
- `updateStatisticsDisplay()` uses class selectors for unique elements - The function selects `.total-books` and `.total-members` using class selectors, even though these elements represent unique values. Once the HTML has been updated, these selectors should be changed to `#total-books` and `#total-members` to improve consistency and clarity.
- `updateStatisticsDisplay()` uses `innerHTML` for plain text - The function assigns numeric values using `innerHTML`. Although the values are currently safe, using `textContent` is more appropriate because it avoids unnecessary HTML parsing and reduces potential security risks.
- `importLibraryData()` has no `try...catch` block around `JSON.parse()` - If invalid JSON is supplied, `JSON.parse()` throws a `SyntaxError`, causing the application to fail without providing any feedback. Wrapping the parsing operation in a `try...catch` block allows errors to be handled gracefully.
- `displayBookDetails()` has no null check - The `findBookByISBN()` function can return null when a matching book cannot be found. Since `displayBookDetails()` immediately accesses `book.title`, the application throws a `TypeError` if the book does not exist. A null check should be performed before accessing the object's properties.

c. Minor

- `handleSearch()` is case-sensitive - The function compares titles using `books[i].title.includes(searchTerm)` without normalising letter case. Consequently, searching for "gatsby" does not match "The Great Gatsby". Both strings should be converted to the same case using `toLowerCase()` or `toUpperCase()` before comparison.
- `handleFilterChange()` does not handle the "All" option - When the user selects All Categories, the selected value is "all", but the filtering logic still attempts to match this value against book categories. Since no books belong to a category named "all", the catalogue becomes empty. The function should explicitly detect this option and return the complete list of books.
- `handleFilterChange()` and `handleSearch()` should use `filter()` - Both functions use traditional for loops and manually push matching books into new arrays. Replacing these loops with the `Array.prototype.filter()` method would produce cleaner, more concise, and more maintainable code.
- `createMemberForm()` dynamically creates a `<form>` element - The `createMemberForm()` function dynamically creates a `<form>` element using `createElement()` and appends it to the DOM with `appendChild()`. However, the generated form does not include the ARIA labels, validation feedback container, or submit handling that the existing static form provides. To maintain consistency and accessibility, the existing `#borrow-form` structure should be reused, or the dynamic form should be generated using a template literal that includes all the required attributes.
- `createMemberForm()` uses `type="text"` for the email field - The email input is created with `type="text"` instead of `type="email"`. Using `type="email"` enables built-in browser validation and provides the appropriate keyboard layout on mobile devices.

- `createMemberForm()` does not include `<label>` elements - The dynamically created form inputs do not include associated `<label>` elements. Like the static HTML form, dynamically generated forms should also meet accessibility standards by providing labels for all input fields.
- `handleBorrowSubmit()` uses `alert()` for feedback - The function displays messages using the browser's `alert()` dialog. Although functional, `alert()` interrupts the user's workflow and provides an inconsistent user experience. Instead, feedback should be displayed in the dedicated `#borrow-feedback` element using appropriate success or error styling.
- `handleBorrowSubmit()` does not reset the form - After a successful borrowing transaction, the member ID and ISBN fields retain their previous values. Calling `borrowForm.reset()` after a successful submission would clear the form and improve usability.
- `exportLibraryData()` returns a raw object instead of a JSON string - Despite its name, `exportLibraryData()` returns a plain JavaScript object rather than a JSON string. Since exported data should be portable and easy to save or transfer, the function should return `JSON.stringify()` instead.

d. Enhancements/ Modernisation Notes

- `renderBookCatalogue()` should use `DocumentFragment` - The function appends each book card directly to the live DOM, causing a reflow after every `appendChild()` operation. Rendering the cards inside a `DocumentFragment` first and appending the fragment only once significantly improves rendering performance, especially for larger catalogues.
- Book cards should include a `data-isbn` attribute - The rendered book cards do not contain a `data-isbn` attribute, making it difficult to determine which book was selected when a card is clicked. Adding

a data-isbn attribute creates a clean connection between the DOM elements and the underlying data model.

- No debounce is implemented for the search functionality - The `handleSearch()` function executes on every keystroke and re-renders the catalogue each time the user types a character. Implementing a debounce of approximately 300 milliseconds would reduce unnecessary rendering operations and improve performance while maintaining a responsive user experience.
- `loadFromLocalStorage()` requires a null check before parsing - When no data has previously been stored, `localStorage.getItem()` returns null. Although `JSON.parse(null)` returns null rather than throwing an exception, assigning null to the books and members variables causes subsequent operations to fail. The function should check whether data exists before parsing and default to empty arrays (`[]`) when no saved data is available.
- `renderBookCatalogue()` should include loading and empty states - When the catalogue contains no books or a search returns no results, the interface displays a blank area with no explanation. An appropriate loading message and an informative empty-state message should be displayed so users understand why no books are visible.